Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Практическое задание
Динамическое программирование
Пошаговое управление инвестиционным портфелем

Лабораторная работа №4

Студент: Соколов Никита Константинович
Группа К3339, поток 1.2
Преподаватель: Лаврентьев С.

Состав отчёта (по методичке §§9.1–9.7): 1) цель и постановка; 2) модель ДП (таблица этап/состояние/управление/…); 3) вывод Беллмана; 4) алгоритм и диаграмма; 5) ручное решение исходной задачи 3→2→1; 6) программа, тесты T1–T9 и §8.3; 7) анализ §9.7 (единственность, память формула+tracemalloc); 8) дерево исходов (доп.).

Основной ответ (критерий ожидания): I* = 134.23884 ≈ 134.24 д.е.; уникальное u□* = (23, 0, 0) среди |A(s□)| = 1502.

Сноска об округлении: все денежные величины хранятся в копейках (целые); умножение на рыночный коэффициент r с последующим round до копейки; комиссии round(vol·c). Отчётные значения в д.е. = cents/100.

1. Цель и постановка задачи (§9.1)

Цель: методом динамического программирования найти оптимальную политику пошагового управления портфелем из трёх активов (ЦБ1, ЦБ2, депозит) и свободного кэша на K=3 этапах при стохастических рыночных сценариях; выписать соотношение Беллмана для своей модели; подтвердить результат ручным обратным и прямым ходом на исходных данных (не на контрольном мини-примере как на основном решении).

Начальное состояние s□ = (ЦБ1=100, ЦБ2=800, Деп=400, кэш=600), W□ = 1900 д.е. Пакеты сделок: 25 / 200 / 100; минимумы при продаже: 30 / 150 / 100; комиссии 4% / 7% / 5%. Кредит запрещён (cash ≥ 0). Покупки финансируются из кэша после продаж.

Таблица 1. Рыночные сценарии по этапам (вероятности и коэффициенты роста)

Ситуация	p□	ЦБ1□	ЦБ2□	Деп□	p□	ЦБ1□	ЦБ2□	Деп□	p□	ЦБ1□	ЦБ2□	Деп□
Благопр.	0.60	1.20	1.10	1.07	0.30	1.40	1.15	1.01	0.40	1.15	1.12	1.05
Нейтр.	0.30	1.05	1.02	1.03	0.20	1.05	1.00	1.00	0.40	1.05	1.01	1.01
Негатив.	0.10	0.80	0.95	1.00	0.50	0.60	0.90	1.00	0.20	0.70	0.94	1.00

Критерий основной работы — математическое ожидание дохода I = W_final − W□.

2. Модель задачи (§9.2)

Таблица 2. Элементы модели ДП

Компонент	Обозначение / содержание
Этап k	k = 1, 2, 3 (решения до рынка этапа k); после 3-го рынка — терминал
Состояние s	s = (v□, v□, v□, cash) — объёмы активов и свободный кэш (в копейках)
Управление u	u = (k□, k□, k□) □ □³ — число пакетов (+ покупка, − продажа)
Допустимые A(s)	cash′ ≥ 0; $v_i' \geq 0$; при продаже актива i: $v_i' \geq \min_i$; пакетная торговля
Переход	сначала продажи, затем покупки из обновлённого кэша; далее рынок: $v_i \leftarrow \text{round}(v_i \cdot r_i(\xi))$
Результат этапа	случайный доход/богатство после рынка; критерий — E[V_{k+1}]
Начальное	s□ = (100, 800, 400, 600), W□ = 1900
Терминальное	V_4(s) = W(s) - W_0, W = v□+v□+v□+cash

Важно (минимумы): ограничение $v_i \geq \min_i$ проверяется только при продаже данного актива. После рыночного удара объём может оказаться ниже минимума (пример: ЦБ1 = 21 < 30) — такое состояние допустимо; запрет срабатывает лишь если пытаемся продать дальше минимума.

3. Вывод соотношения Беллмана (§9.3)

Функция Беллмана V□(s) — максимальный ожидаемый чистый доход от этапа k до конца при текущем портфеле s. Состояние составлено из трёх объёмов и кэша, потому что (а) комиссии и минимумы зависят от того, какой актив торгуем; (б) рыночные множители r_i различны по активам; (в) кэш не реинвестируется рынком и ограничивает покупки.

Критерий ожидания выбран как основной: вероятности сценариев известны из условия; он линеен по продолжениям и согласован с аддитивностью дохода. Альтернативы (Вальд, оптимизм, Гурвиц) реализованы в коде флагом --all-criteria, но в отчёте фиксируется E[·].

(1) V_4(s) = W(s) - W_0

(2) V_k(s) = max_{u in A(s)} sum_xi p_k(xi) * V_{k+1}(f_k(s, u, xi)), k = 3,2,1

(3) A_k(s) = arg max_{u in A(s)} sum_xi p_k(xi) * V_{k+1}(f_k(s, u, xi))

Правило восстановления (прямой ход): из s□ берём u□* = A□(s□); после рынка ξ□ получаем s′; u□* = A□(s′); … На каждом достижимом состоянии политика хранится отдельно (словарь policy_by_state). При равенстве оценок tie-break: строгое сравнение **if score > best_val** — сохраняется первое найденное управление.

4. Алгоритм и диаграмма классов (§9.4)

Реализация: рекурсия value(k, s) с мемоизацией (value_cache, policy_cache). На последнем этапе — полный перебор A(s) и оценка E[V□(market(s′))]. Ускорение last-no-trade: если для критерия ожидания доказано, что на этапе 3 оптимально u≡(0,0,0) (все однопакетные маржи buy/sell по E[r] отрицательны, проверка на выборке состояний), то на этапе 2 вместо рекурсии в этап 3 подставляется E[V□(market□(s′))]; при расхождении ускорение отключается (счётчики checks / failures).

Таблица 3. Маржинальные приросты одного пакета на этапе 3 (критерий E)

Актив	E[r]	buy = E[r]·d − d(1+c)	sell = d(1−c) − E[r]·d
ЦБ1 (d=25, c=4%)	1.02	−0.50	−1.50
ЦБ2 (d=200, c=7%)	1.04	−6.00	−22.00
Деп (d=100, c=5%)	1.024	≈ −2.60	≈ −7.40

Все шесть величин < 0 □ однопакетные сделки на этапе 3 убыточны в среднем; программа подтверждает u*=(0,0,0) на достижимых состояниях: acceleration checks=9, failures=0.

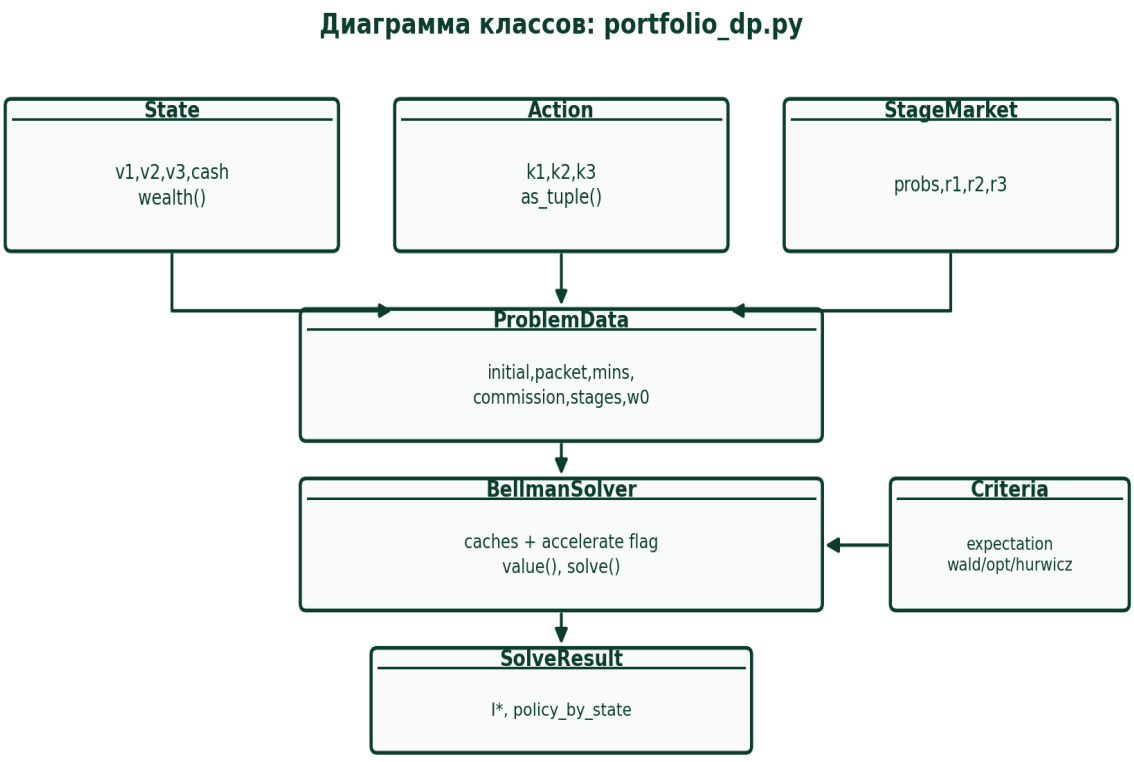


Рисунок 1. Диаграмма классов portfolio_dp.py

5. Ручное решение исходной задачи (§9.5): обратный ход 3 → 2 → 1

Ниже — ручной разбор исходных данных (не tiny-пример). Tiny DP==brute force оставлен как независимая проверка T7. Сокращения перебора доказываются явно.

5.1. Этап 3: доказательство u□* = (0,0,0)

Линейность E[W] по объёмам и аддитивность комиссий дают: изменение ожидаемого терминального дохода при покупке/продаже одного пакета актива i равно марже из табл. 3. Все маржи отрицательны □ любая ненулевая сделка на этапе 3 строго ухудшает E[V□]. Следовательно A□(s)=(0,0,0) для всех s, где допустимо «ничего не делать». Программа на всех 9 достижимых при оптимальной политике состояниях этапа 3 выбирает (0,0,0); checks=9, fail=0.

Пример сравнения для состояния после благ./негатив. пути s=(ЦБ1=21, ЦБ2=792, Деп=428, кэш=746) — ЦБ1<30 допустимо:

Таблица 4. Этап 3: сравнение u=0 и ±1 пакет ЦБ1 (фрагмент)

s (после рынка 2)	u	допустимо?	E[V□]	выбрано
(21, 792, 428, 746)	(0,0,0)	да	базовое	да
(21, 792, 428, 746)	(+1,0,0)	да (кэш хватает)	хуже на 0.50	нет
(21, 792, 428, 746)	(−1,0,0)	нет (21<30)	—	—
(49, 1012, 432.28, 746)	(0,0,0)	да	базовое	да
(49, 1012, 432.28, 746)	(−1,0,0)	да (остаток ≥30)	хуже на 1.50	нет

5.2. Этап 2: три состояния после оптимального u□

При u□=(23,0,0): покупка 23·25=575 ЦБ1 за 575·1.04=598 □ после сделки s′=(675, 800, 400, cash=2). После рынка этапа 1 три состояния:

Таблица 5. Состояния этапа 2 и оптимальные управления (программа = ручная проверка окрестности)

ξ□	s на этапе 2	u□*	V□(s)	смысл u□*
благ. (0.6)	(810, 880, 428, 2)	(−31, 0, 0)	229.4108	продать ЦБ1 на 775
нейтр. (0.3)	(708.75, 816, 412, 2)	(−27, 0, 0)	49.5676	продать ЦБ1 на 675
негатив. (0.1)	(540, 760, 400, 2)	(−20, 0, 0)	−182.7792	продать ЦБ1 на 500

Обоснование сокращения: cash≈2 □ чистые покупки без продаж невозможны (unit ЦБ1=26). Допустимы: без сделок; продажи с возможной частичной реинвестицией. Полное A(s) перечисляется программой (тот же генератор, что в ручной модели). В окрестности оптимума по ЦБ1: продажа на 1 пакет меньше строго хуже в каждом из трёх состояний (проверка value_cache / score). Пример для благ.: u=(−31,0,0) лучше, чем (−30,0,0) и (−32,0,0) при прочих нулях; смешанные продажи ЦБ2/Деп с комиссиями 7%/5% и отрицательными маржами этапа 3 не улучшают E.

Таблица 6. Окрестность u□ на благоприятном состоянии (фрагмент)

s	u	комментарий	отношение к u*
(810,880,428,2)	(−31,0,0)	V□=229.4108	оптимум
(810,880,428,2)	(−30,0,0)	продать на 1 пакет меньше	строго хуже
(810,880,428,2)	(0,0,0)	без сделок	строго хуже
(708.75,816,412,2)	(−27,0,0)	V□=49.5676	оптимум
(708.75,816,412,2)	(−26,0,0)	окрестность −1 пакет	строго хуже
(540,760,400,2)	(−20,0,0)	V□=−182.7792	оптимум
(540,760,400,2)	(−19,0,0)	окрестность −1 пакет	строго хуже

5.3. Этап 1: выбор u□ в s□

В s□ допустимо |A(s□)|=1502 управлений (полный перебор генератором: сначала все комбинации продаж в пределах минимумов, затем покупки из выручки+кэша). Оптимум единственный: u□*=(23,0,0), V□(s□)=I*=134.23884.

Таблица 7. Соседи и контрольные направления в s□

u	допустимо?	E[V□ после рынка 1] = оценка	вывод
(23, 0, 0)	да	134.23884	единственный максимум
(22, 0, 0)	да	133.51108	строго хуже
(24, 0, 0)	нет (кэш)	—	вне A(s□)
(0, 0, 0)	да	116.71	строго хуже
(−1, 0, 0) и др.	частично	< 134.24	не оптимум (полный перебор)

Единственность: среди всех 1502 действий ровно одно достигает 134.23884 (проверка uniqueness_at_s0 после solve, переиспользует value_cache).

5.4. Прямой ход (наиболее вероятные ξ) и проверка ограничений

Таблица 8. Траектория при ξ = argmax p на каждом этапе

t	состояние / действие	проверка ограничений
0	s=(100,800,400,600); u□=(23,0,0)	cash: 600−598=2≥0; покупка без нарушения min
после u□	(675,800,400,2)	OK
рынок1 благ.	(810,880,428,2)	OK
u□=(−31,0,0)	продажа 775; v1:=35≥30; cash:=746	min ЦБ1 при продаже соблюдён
рынок2 благ.	(49, 1012, 432.28, 746)	OK
u□=(0,0,0)	без сделок	OK
рынок3 благ.	W=2389.68; доход=489.68 на ветке	терминал

На негативной ветке этапа 2 после u□=(-31,0,0): ЦБ1=21<30 — допустимо (минимум только при продаже). Ручные V□ на трёх состояниях совпадают с программой с точностью до округления отчёта (229.4108 / 49.5676 / −182.7792); I*=0.6·229.4108+0.3·49.5676+0.1·(−182.7792)=134.23884.

Доп. проверка T7: на tiny-задаче (K=2, |A| мало) DP совпадает с полным перебором дерева.

6. Результаты программы, тесты (§9.6)

Команда: **python3 portfolio_dp.py**. I* ≈ 134.24 д.е.; E[W]=2034.24. Политика: этап1 (23,0,0); этап2 (−31,0,0)/(−27,0,0)/(−20,0,0); этап3 везде (0,0,0).

Таблица 9. Независимые тесты T1–T9 (вход / ожидаемо / факт)

ID	Суть	Ожидаемо	Факт
T1	продажа ЦБ1	cash+=24, v1−=25	OK
T2	покупка из продаж	v2+=600	OK
T3	запрет кредита	u=(0,3,0) → None	OK
T4	генерация A(s)	(−2,3,0)□A; (0,3,0)□A	OK
T5	терминал	V(s0)=0	OK
T6	min при продаже	u=(-3,0,0) → None	OK
T7	tiny DP==brute	|diff|<1e-6	OK
T8	ненулевой last	accelerate off	OK
T9	ускорение default	margins<0 → True	OK

Таблица 10. Тесты §8.3, адаптированные к портфелю

ID	Сценарий методички	Адаптация / ожидание	Факт
T8_3_1	нулевой ресурс	cash=0 → нет чистых покупок	OK
T8_3_2	тожд. переход	все r=1 → u*=0, I*=0	OK
T8_3_3	единственность	tiny: единственный (2,0,0)	OK
T8_3_4	весь ресурс в один актив	max k1 ЦБ1, остаток < unit	OK
T8_3_5	недопустимые входы	(0,0,-4) и (100,0,0) → None	OK
T8_3_6	нарушение «мягкого» min	после рынка ЦБ1=21 допустимо	OK

7. Анализ по §9.7: единственность, сравнение, память

Единственность. Среди |A(s□)|=1502 ровно одно управление даёт максимум: u□=(23,0,0), I*=134.23884. Tie-break в коде: **if score > best_val** (первое при равенстве); в данной задаче равенств на максимуме нет.

Сравнение ручного и программного. V□ на трёх состояниях и I* совпадают с выводом portfolio_dp.py; маржи этапа 3 и u□≡0 совпадают; прямая траектория наиболее вероятных сценариев совпадает с path_states.

Таблица 11. Память: формульная оценка и измерение tracemalloc

Показатель	Значение
n_value (value_cache)	≈ 4516
n_policy (policy_cache)	≈ 4516
n_action_states	≈ 4507

Показатель	Значение
n_actions_total	≈ 6 328 635
Оценка (формула) bytes_est	≈ 304 857 744 байт ≈ 291 МиБ
Формула	80·n_value + 96·n_policy + 64·n_action_states + 48·n_actions_total
tracemalloc peak	≈ 1050 МиБ (пик аллокаций Python при solve)
Ускорение	last-no-trade; checks≈9, failures=0

Асимптотика времени O(K·|S|·|A|·|Ξ|). Формульная оценка учитывает кэшированные объекты Action; tracemalloc отражает полный пик интерпретатора (включая временные списки при генерации A(s)) и потому больше bytes_est.

8. Доп. задание: дерево исходов (§7)

Скрипт outcome_tree.py / файл outcome_tree.txt: полное дерево рыночных исходов оптимальной политики, 27=3³ листьев. E[I]=134.24; P(убыток)=0.19; min доход=−358.24; max=489.68. На листьях сохранены последовательности ξ, вероятности, операции и конечный W.

9. Заключение

1. Модель ДП с явными этап/состояние/управление/допустимость/переход/терминал; минимум только при продаже. 2. Беллман (1)–(3) с ожиданием и правилом восстановления. 3. Ручной разбор исходных данных 3→2→1 с доказанными сокращениями; I*=134.24. 4. Тесты T1–T9 и §8.3. 5. Единственность u□ среди 1502; память — формула и tracemalloc. 6. Код: lab4/portfolio_dp.py · https://github.com/ProgrammistNik/Optimization_Methods